

```

module Main where import Data.List (nub, sort) import Grammars import Interp import Lexer
import MiniLispPlusPlus (evalua) import Test.QuickCheck -- Generadores -----
----- newtype Programa = Programa ASA deriving (Show) newtype
ListaNat = ListaNat [Int] deriving (Show) newtype Nombres = Nombres [String] deriving (Show)
ids :: [String] ids = ["x", "y", "z", "u", "v", "dato", "aux"] genArgs :: Int -> Gen [ASA] genArgs n =
do k <- chooseInt (2, 4) vectorOf k (genASA (n `div` 2)) genBindings :: Int -> Gen [Binding]
genBindings n = do k <- chooseInt (1, 3) xs <- vectorOf k (elements ids) es <- vectorOf k
(genASA (n `div` 2)) pure (zip xs es) genASA :: Int -> Gen ASA genASA 0 = oneof [ Num .
getNonNegative <$> arbitrary, Boolean <$> arbitrary, Id <$> elements ids ] genASA n =
frequency [ (5, genASA 0), (1, And <$> genArgs n), (1, Or <$> genArgs n), (2, Add <$> genArgs
n), (2, Sub <$> genArgs n), (2, Mul <$> genArgs n), (2, Div <$> genArgs n), (1, Lt <$> genArgs
n), (1, Gt <$> genArgs n), (1, Le <$> genArgs n), (1, Ge <$> genArgs n), (1, Expt <$> genASA
(n `div` 2) <*> genASA (n `div` 2)), (1, EqP <$> genASA (n `div` 2) <*> genASA (n `div` 2)), (1,
Not <$> genASA (n `div` 2)), (1, Add1 <$> genASA (n `div` 2)), (1, Sub1 <$> genASA (n `div`
2)), (1, ZeroP <$> genASA (n `div` 2)), (1, Let <$> genBindings n <*> genASA (n `div` 2)), (1,
LetStar <$> genBindings n <*> genASA (n `div` 2)) ] instance Arbitrary Programa where
arbitrary = sized (fmap Programa . genASA . min 8) shrink _ = [] instance Arbitrary ListaNat
where arbitrary = do k <- chooseInt (2, 6) ListaNat <$> vectorOf k (chooseInt (0, 30)) instance
Arbitrary Nombres where arbitrary = do k <- chooseInt (0, 20) Nombres <$> vectorOf k
(elements ids) -- Referencias pequenas e invariantes ----- normaliza ::
[String] -> [String] normaliza = sort . nub fvRef :: ASA -> [String] fvRef (Id x) = [x] fvRef (Num _)
= [] fvRef (Boolean _) = [] fvRef (And es) = concatMap fvRef es fvRef (Or es) = concatMap fvRef
es fvRef (Add es) = concatMap fvRef es fvRef (Sub es) = concatMap fvRef es fvRef (Mul es) =
concatMap fvRef es fvRef (Div es) = concatMap fvRef es fvRef (Lt es) = concatMap fvRef es
fvRef (Gt es) = concatMap fvRef es fvRef (Le es) = concatMap fvRef es fvRef (Ge es) =
concatMap fvRef es fvRef (Expt e1 e2) = fvRef e1 ++ fvRef e2 fvRef (EqP e1 e2) = fvRef e1 ++
fvRef e2 fvRef (Not e) = fvRef e fvRef (Add1 e) = fvRef e fvRef (Sub1 e) = fvRef e fvRef (ZeroP
e) = fvRef e fvRef (Let bs body) = concatMap (fvRef . snd) bs ++ filter (`notElem` map fst bs)
(fvRef body) fvRef (LetStar [] body) = fvRef body fvRef (LetStar ((x, e) : bs) body) = fvRef e ++
filter (/= x) (fvRef (LetStar bs body)) namesRef :: ASA -> [String] namesRef (Id x) = [x] namesRef
(Num _) = [] namesRef (Boolean _) = [] namesRef (And es) = concatMap namesRef es
namesRef (Or es) = concatMap namesRef es namesRef (Add es) = concatMap namesRef es
namesRef (Sub es) = concatMap namesRef es namesRef (Mul es) = concatMap namesRef es
namesRef (Div es) = concatMap namesRef es namesRef (Lt es) = concatMap namesRef es
namesRef (Gt es) = concatMap namesRef es namesRef (Le es) = concatMap namesRef es
namesRef (Ge es) = concatMap namesRef es namesRef (Expt e1 e2) = namesRef e1 ++
namesRef e2 namesRef (EqP e1 e2) = namesRef e1 ++ namesRef e2 namesRef (Not e) =
namesRef e namesRef (Add1 e) = namesRef e namesRef (Sub1 e) = namesRef e namesRef
(ZeroP e) = namesRef e namesRef (Let bs body) = map fst bs ++ concatMap (namesRef . snd)
bs ++ namesRef body namesRef (LetStar bs body) = map fst bs ++ concatMap (namesRef .
snd) bs ++ namesRef body parsea :: String -> ASA parsea = parse . lexer -- Reto 1: lexer -----
----- prop_lexer_let :: Bool prop_lexer_let = lexer "(let* ((x 2)
(y (+ x 1))) (+ x y))" == [ TokenPA, TokenLetStar, TokenPA, TokenPA, TokenId "x", TokenNum 2,
TokenPC, TokenPA, TokenId "y", TokenPA, TokenSuma, TokenId "x", TokenNum 1, TokenPC,
TokenPC, TokenPC, TokenPA, TokenSuma, TokenId "x", TokenId "y", TokenPC, TokenPC ]
prop_lexer_prioridad :: Bool prop_lexer_prioridad = lexer "let let* letra let1" == [ TokenLet,
TokenLetStar, TokenId "letra", TokenId "let1" ] -- Reto 2: parser -----
----- prop_parse_lets :: Bool prop_parse_lets = and [ parsea "(let ((x 2) (y 3)) (+ x y))"
== Let [("x", Num 2), ("y", Num 3)] (Add [Id "x", Id "y"]), parsea "(let* ((x 2) (y (+ x 1))) (+ x y))" ==
LetStar [("x", Num 2), ("y", Add [Id "x", Num 1])] (Add [Id "x", Id "y"]) ] -- Reto 3: sustitucion
nominal ----- prop_free_vars :: Programa -> Bool
prop_free_vars (Programa e) = normaliza (freeVars e) == normaliza (fvRef e) prop_names ::
Programa -> Bool prop_names (Programa e) = normaliza (names e) == normaliza (namesRef e)
prop_fresh_name :: Nombres -> Bool prop_fresh_name (Nombres usados) = freshName
usados `notElem` usados prop_sust_identidad :: Programa -> Property prop_sust_identidad
(Programa s) = forAll (elements ids) $ \x -> sust (Id x) x s == s prop_sust_alcance :: Bool

```

```

prop_sust_alcance = sust (Let [("x", Id "x")] (Add [Id "x", Id "z"])) "x" (Num 4) == Let [("x", Num
4)] (Add [Id "x", Id "z"]) prop_sust_evitar_captura :: Bool prop_sust_evitar_captura = case sust
(Let [("y", Num 0)] (Add [Id "x", Id "y"])) "x" (Id "y") of Let [(z, Num 0)] (Add [Id "y", Id z']) -> z ==
z' && z /= "y" && normaliza (freeVars (Let [(z, Num 0)] (Add [Id "y", Id z']))) == ["y"] _ -> False
prop_sust_letstar_alcance :: Bool prop_sust_letstar_alcance = sust ( LetStar [ ("x", Id "x"), ("y",
Add [Id "x", Id "z"]) ] (Add [Id "x", Id "z"]) ) "x" (Num 4) == LetStar [ ("x", Num 4), ("y", Add [Id "x",
Id "z"]) ] (Add [Id "x", Id "z"]) prop_sust_letstar_evitar_captura :: Bool
prop_sust_letstar_evitar_captura = case sust (LetStar [("y", Num 0)] (Add [Id "x", Id "y"])) "x" (Id
"y") of LetStar [(z, Num 0)] (Add [Id "y", Id z']) -> z == z' && z /= "y" && normaliza (freeVars
(LetStar [(z, Num 0)] (Add [Id "y", Id z']))) == ["y"] _ -> False prop_sust_many_simultanea :: Bool
prop_sust_many_simultanea = sustMany (Add [Id "x", Id "y"]) [("x", Id "y"), ("y", Num 2)] == Add
[Id "y", Num 2] -- Reto 4: semantica operacional de paso grande -----
prop_bigstep_base_y_primitivas :: Bool prop_bigstep_base_y_primitivas = and [ bigStep (Num
0) == Just (Num 0), bigStep (Boolean False) == Just (Boolean False), bigStep (Id "x") ==
Nothing, bigStep (And [Boolean True, Boolean True, Boolean False]) == Just (Boolean False),
bigStep (Or [Boolean False, Boolean True, Boolean False]) == Just (Boolean True), bigStep
(Add [Num 1, Num 2, Num 3]) == Just (Num 6), bigStep (Sub [Num 10, Num 3, Num 9]) == Just
(Num 0), bigStep (Mul [Num 2, Num 3, Num 4]) == Just (Num 24), bigStep (Div [Num 24, Num
3, Num 2]) == Just (Num 4), bigStep (Lt [Num 1, Num 2, Num 4]) == Just (Boolean True),
bigStep (Gt [Num 5, Num 3, Num 4]) == Just (Boolean False), bigStep (Le [Num 1, Num 1, Num
2]) == Just (Boolean True), bigStep (Ge [Num 3, Num 3, Num 1]) == Just (Boolean True),
bigStep (Expt (Num 2) (Num 5)) == Just (Num 32), bigStep (EqP (Boolean True) (Boolean
True)) == Just (Boolean True), bigStep (EqP (Num 2) (Num 3)) == Just (Boolean False), bigStep
(EqP (Num 2) (Boolean True)) == Nothing, bigStep (Not (Num 0)) == Just (Boolean False),
bigStep (Not (Boolean False)) == Just (Boolean True), bigStep (Add1 (Num 8)) == Just (Num 9),
bigStep (Sub1 (Num 0)) == Just (Num 0), bigStep (ZeroP (Num 0)) == Just (Boolean True),
bigStep (Div [Num 8, Num 0]) == Nothing, bigStep (Add [Boolean True, Num 1]) == Nothing,
bigStep (Add [Add [Num 1, Num 2], Mul [Num 3, Num 4]]) == Just (Num 15) ]
prop_bigstep_suma_naria :: ListaNat -> Bool prop_bigstep_suma_naria (ListaNat ns) = bigStep
(Add (map Num ns)) == Just (Num (sum ns)) prop_let_paso_grande :: Bool
prop_let_paso_grande = and [ bigStep ( Let [ ("x", Add [Num 1, Num 2]), ("y", Add [Num 3, Num
4]) ] (Add [Id "x", Id "y"]) ) == Just (Num 10), bigStep ( Let [("x", Num 2)] (Let [("y", Add [Id "x",
Num 1])]) (Add [Id "x", Id "y"]) ) == Just (Num 5) ] prop_let_simultaneo :: ListaNat -> Bool
prop_let_simultaneo (ListaNat (n : m : _)) = bigStep (Let [("x", Num n), ("y", Num m)] (Add [Id
"x", Id "y"])) == Just (Num (n + m)) prop_let_simultaneo _ = False prop_letstar_secuencial ::
NonNegative (Small Int) -> NonNegative (Small Int) -> Bool prop_letstar_secuencial
(NonNegative (Small n)) (NonNegative (Small m)) = bigStep ( LetStar [("x", Num n), ("y", Add [Id
"x", Num m])] (Add [Id "x", Id "y"]) ) == Just (Num (2 * n + m)) prop_letstar_caso_base :: Bool
prop_letstar_caso_base = bigStep (LetStar [] (Add [Num 1, Num 2])) == Just (Num 3)
prop_letstar_sombreado :: NonNegative (Small Int) -> Bool prop_letstar_sombreado
(NonNegative (Small n)) = bigStep ( LetStar [("x", Num n), ("x", Add [Id "x", Num 1])] (Id "x") ) ==
Just (Num (n + 1)) prop_distingue_let_letstar :: Bool prop_distingue_let_letstar = let bs = [("x",
Num 2), ("y", Add [Id "x", Num 1])] body = Add [Id "x", Id "y"] in bigStep (Let bs body) == Nothing
&& bigStep (LetStar bs body) == Just (Num 5) prop_evalua_texto :: Bool prop_evalua_texto =
and [ evalua "(+ 1 2 3 4)" == Num 10, evalua "(let ((x 2) (y 3)) (* x y))" == Num 6, evalua "(let* ((x
2) (y (+ x 1))) (+ x y))" == Num 5, evalua "(let* ((x 2) (x (add1 x))) x)" == Num 3 ] prop_bloqueos
:: Bool prop_bloqueos = and [ bigStep (Add [Boolean True, Num 1]) == Nothing, bigStep (Div
[Num 4, Num 0]) == Nothing, bigStep ( Let [("x", Num 2), ("y", Add [Id "x", Num 1])] (Add [Id "x",
Id "y"]) ) == Nothing, bigStep (Let [("x", Num 1), ("x", Num 2)] (Id "x")) == Nothing ] -- Ejecucion -
----- main :: IO () main = do putStrLn "Reto 1: lexer"
quickCheck prop_lexer_let quickCheck prop_lexer_prioridad putStrLn "Reto 2: parser"
quickCheck prop_parse_lets putStrLn "Reto 3: sustitucion nominal sin captura" quickCheckWith
stdArgs {maxSuccess = 150} prop_free_vars quickCheckWith stdArgs {maxSuccess = 150}
prop_names quickCheckWith stdArgs {maxSuccess = 150} prop_fresh_name quickCheckWith
stdArgs {maxSuccess = 100} prop_sust_identidad quickCheck prop_sust_alcance quickCheck
prop_sust_evitar_captura quickCheck prop_sust_letstar_alcance quickCheck
prop_sust_letstar_evitar_captura quickCheck prop_sust_many_simultanea putStrLn "Reto 4:

```

semantica operacional de paso grande" quickCheck prop_bigstep_base_y_primitivas
quickCheckWith stdArgs {maxSuccess = 200} prop_bigstep_suma_naria quickCheck
prop_let_paso_grande quickCheckWith stdArgs {maxSuccess = 150} prop_let_simultaneo
quickCheckWith stdArgs {maxSuccess = 150} prop_letstar_secuencial quickCheck
prop_letstar_caso_base quickCheckWith stdArgs {maxSuccess = 150} prop_letstar_sombreado
quickCheck prop_distingue_let_letstar quickCheck prop_evalua_texto quickCheck
prop_bloqueos